BATON

A Protocol for Resumable, Verifiable AI Agent Work

TECHNICAL WHITEPAPER / VERSION 1.0 / 08 NOVEMBER 2026

Architecture, authority, encrypted recovery, payments, security and evidence

REFERENCE IMPLEMENTATION
github.com/noMathematician1729/baton

Status note: architecture and implementation description, not a claim of a completed production deployment.

Abstract

BATON is a protocol-oriented system for continuing an AI agent job when its current worker becomes unavailable. Rather than equating progress with one agent process, BATON separates the job authority, the accepted checkpoint, the worker runtime and the money spent on external services. A successor worker can recover a buyer-approved state, obtain narrowly scoped access and resume without repeating a previously settled purchase where authoritative receipts and output are available.

The reference implementation combines Ethereum Sepolia escrow and EIP-712 authorization, an ENS-scoped authority path, encrypted checkpoint storage intended for Arkiv, a buyer-host SQLite payment ledger, a Hedera x402-capable provider adapter and a worker recovery runtime. The system deliberately retains a trusted buyer coordinator. It does not assert fully trustless worker election, Byzantine fault tolerance or recovery after the loss of all buyer-controlled infrastructure.

1. Motivation and problem definition

Long-running agent work spans context windows, machines, paid tools and execution environments. A process crash or operator departure can strand partially completed work. Naive retry often loses context, repeats paid requests, creates inconsistent results or permits the former worker to continue writing after reassignment. Conventional task orchestration can restart a process, but it does not by itself establish portable economic authority, cryptographically bound accepted progress or worker-independent retrieval of purchased output.

BATON targets the narrower but consequential failure mode of worker discontinuity: worker A may stop while a buyer-controlled job remains valid, and worker B should continue from the last accepted boundary with no ambiguity about ownership, budget or reward credits.

1.1 Design objectives

• Portable accepted state: a successor retrieves exactly the checkpoint approved by the buyer.

• Fenced authority: a monotonically advancing assignment epoch makes stale worker actions rejectable.

• Economic continuity: previously earned credits survive reassignment, and paid operations use stable identities for reconciliation.

• Minimal disclosure: checkpoints and paid outputs are encrypted; keys are granted to the authorized successor.

• Auditable boundaries: chain confirmations, accepted hashes, signed messages and settlement records are independently inspectable within their respective trust domains.

1.2 Non-goals

BATON is not a general decentralized marketplace, an autonomous verifier of arbitrary agent output, a distributed replacement for every local database or a guarantee of exactly-once execution at an external provider. Its MVP does not eliminate the buyer, gateway or key service as trusted availability dependencies.

2. Actors and trust domains

Actor / component	Responsibility and trust
Buyer / coordinator	Funds job, controls initial assignment and acceptance, operates policy and key-grant decisions. Trusted for liveness and acceptance policy.
Worker A / B	Executes job, proposes checkpoints, authenticates requests, and (for B) proves recovery of the accepted head. Workers are not trusted to self-assign.
Sepolia escrow	Canonical job assignment, epoch, accepted checkpoint commitment, milestone credits, withdrawals and refunds.
ENS authority path	Scoped worker authorization through ENS resolver and BATON authority adapter; distinct deployed addresses.
Arkiv content store	Intended encrypted manifest/artifact/output persistence. Availability and retention are external dependencies.
Gateway + SQLite	Single-writer payment admission, reservations, reconciliation, idempotency metadata and references.
Hedera paid provider	External paid-operation submission and authoritative status/output lookup, once configured.

3. System architecture

BATON separates control plane, data plane and payment plane. The control plane decides who may work and which checkpoint is accepted. The data plane carries encrypted state and artifacts. The payment plane tracks tool purchases separately from milestone rewards. This separation prevents a worker-local filesystem or a local payment row from becoming the sole source of truth for an accepted handoff.

Plane	Canonical records	Main interfaces
Authority	Escrow job, worker, epoch, accepted head/locator, credits	Solidity escrow, ENS adapter, EIP-712 signatures
Recovery data	Encrypted manifest, ordered chunks, adapter state, purchased output	Arkiv adapter, key service, worker recovery

Plane	Canonical records	Main interfaces
Payments	Logical operation, reservation, settlement state, output reference	SQLite gateway, Hedera x402 provider adapter
Execution	Worker capabilities, heartbeat, checkpoint proposal, takeover acknowledgement	Worker runtime, HTTP API, MCP tools

3.1 Logical execution flow

Fund → authorize → execute → stage encrypted checkpoint → validate → accept and credit → detect interruption → drain and reconcile → recover B → acknowledge accepted head → confirm takeover → resume. The order matters: a staged blob is not an accepted checkpoint, a timeout is not proof of death, and a pending provider operation is not safely repeatable until reconciled.

4. Job and checkpoint model

A job binds a buyer, task identity, reward budget, acceptance rules and authorized worker assignment. An accepted checkpoint identifies a specific serialized encrypted manifest envelope, its storage locator, parent and sequence. Acceptance-spec commitments describe what counts as valid work; they must not be confused with the digest of the actual runtime state.

4.1 Manifest and commitment

A recoverable package can include job/task identifiers, schema version, assignment epoch, checkpoint sequence, parent accepted head, producer metadata, runtime adapter identifier, encrypted state, artifact list, chunk order, lengths and cryptographic digests. The outer commitment is computed over exact versioned bytes, while inner encryption digests have separately defined semantics. The locator and head are bound into the signed milestone offer after the envelope is staged, avoiding self-referential hashing.

The coordinator validates the candidate, its lineage, policy and required artifacts before accepting the new head. The canonical accepted-head transition and milestone credit are designed as one contract-level transition. Upload alone confers neither acceptance nor payment.

4.2 State adapters

The reference runtime includes deterministic and Beta-Bernoulli quantitative state adapters. A recovery adapter must validate schema compatibility and reconstruct execution-relevant state, not merely provide a textual summary. A generic LLM context reconstruction is not equivalent to bitwise or semantic resumption of every tool environment.

5. Identity, authorization and replay protection

BATON uses EIP-712 typed-data signing for actions bound to the chain, verifying contract, action type, job/task, worker epoch, nonce or sequence, payload commitment and expiry where applicable. The server checks recovered signer identity and current authority; an address supplied in JSON is not sufficient authentication. Replayed, expired, cross-job or stale-epoch messages must fail.

The ENS path supplies scoped grants used by the authority adapter. The ENS resolver and BATON adapter are separate contracts. Initial assignment remains buyer-controlled and requires an appropriate job-scoped grant; the current contract interface does not demand a separate worker consent signature. A worker grant does not confer buyer powers.

5.1 Epoch fencing

Each successful takeover advances the assignment epoch. Gateway requests, checkpoint offers and recovery acknowledgements are checked against the confirmed assignment and epoch. This is a logical fencing mechanism: it prevents newly admitted stale-worker actions, but cannot retroactively cancel a provider request already submitted before the fence. Such requests must be reconciled.

6. Escrow, milestones and reward accounting

Milestone rewards are escrowed on Ethereum Sepolia using an ERC-20 test asset in the demonstrated lifecycle. The escrow tracks funded value, accepted milestones, worker credits, withdrawals and buyer refunds. A checkpoint offer signs the accepted checkpoint identity, locator and sequence, so credit is not earned for a disconnected or ambiguous runtime head.

The revised V2 design uses cumulative withdrawn credits rather than a single claimed flag. This permits a worker to withdraw after an early milestone and again after earning additional credit. Earned credit persists across takeover and closure. A withdrawal is a worker-signed claim transaction; the relay checks chain, escrow address, exact calldata, value and recovered sender.

6.1 Accounting invariant

For a funded job, the intended conservation relationship is: funded = unallocated + outstanding worker credits + outstanding buyer refunds + total withdrawn, with all categories defined consistently against contract state. Reward escrow is distinct from paid-tool spending; a 3/3/4-token milestone example is not a Hedera tool-payment breakdown.

7. Paid tool operations and Hedera x402

The gateway derives a stable logical fingerprint for each paid operation, checks task admission and budget, reserves a durable ledger entry and then submits to the configured provider. Settlement can be settled, not settled or unknown. An unknown result must not be treated as a failed payment and retried blindly. After restart or takeover, the gateway reconciles provider status and retrieves the original output when possible.

The reference code includes a configurable Hedera x402 submit/status/output client and policy fixtures. However, the repository status reviewed for this paper reported missing real provider endpoints, with the local payment fixture selected. Therefore a real Hedera settlement, actual debit and authoritative purchased-output lookup were not demonstrated in that evidence baseline.

7.1 Idempotency is conditional

BATON can avoid duplicate purchase only if the provider supports a stable operation identity or authoritative settlement/output lookup, and if the gateway preserves and reconciles its durable records. The design does not claim universal exactly-once execution for arbitrary external services. Output availability and settlement confirmation are distinct checks.

8. Interruption detection and takeover

The opt-in buyer-side policy runner monitors authenticated heartbeats for the current worker and epoch. A timeout is eligible only after a valid heartbeat has been observed; missing heartbeats alone are not definitive proof of failure. When the configured threshold is exceeded, the runner closes local admissions, drains in-flight submissions and reconciles reserved, submitted or unknown operations. Unresolved payment states keep the job closed.

B obtains a buyer-authorized key grant for the exact accepted head, fetches and validates the package, and signs an acknowledgement binding that head and current policy/epoch. The buyer submits the takeover transaction, waits for configured Sepolia confirmations, re-reads canonical assignment and epoch, and only then reopens work under B. If A wins a checkpoint-acceptance race during drain, B must acknowledge the updated accepted head before takeover proceeds.

8.1 Failure matrix

Failure	Expected safe response
A crashes before staging	Resume from prior accepted checkpoint; unaccepted local progress may be lost.
A stages but buyer does not accept	Do not promote staged bytes or award credit.
Provider response is lost	Preserve unknown state and reconcile; do not blindly repurchase.
A submits after B takeover	Reject stale epoch/assignment at admission and contract checks.
B has wrong key or artifact bytes	Fail recovery and keep task closed.
Buyer gateway or key service fails	Autonomous recovery is not guaranteed; restore buyer infrastructure.
Arkiv data expires or is unavailable	Recovery may fail; no independent content mirror is claimed.

9. Encryption, key grants and data boundaries

Checkpoints, manifests, artifacts and paid outputs are intended to be encrypted before persistence. The buyer-host key service grants decryption material to a designated worker for a specific job, task and accepted checkpoint. The key-grant request is buyer-signed and bound to the worker and key identity, with nonce consumption and confirmed authority checks. A TLS client and encrypted key transport support remote retrieval.

SQLite stores payment-operation metadata, not checkpoint payloads, and is not replicated to worker machines. The buyer-host gateway remains the only ledger writer. Arkiv is the intended content layer; it is not a replacement for escrow authority or for the gateway’s local transactional coordination.

10. Security model and limitations

10.1 Protected properties

• Authorization integrity through typed signatures, ENS-scoped grants and confirmed escrow state.

• Replay and stale-worker resistance through nonces, expiry, task binding and epoch fencing.

• Checkpoint integrity through exact byte commitments, lineage checks and artifact digest verification.

• Budget safety through durable reservations, per-operation checks and fail-closed reconciliation.

• Confidentiality against unauthorized storage readers, assuming keys and cryptographic implementation remain secure.

10.2 Trust assumptions and residual risks

The buyer is trusted to accept valid work, run the policy, maintain signing keys and grant decryption access. The key service and SQLite gateway are single-host dependencies. Arkiv availability, paid-provider settlement semantics, RPC consistency, worker runtime isolation and correct off-chain verification remain necessary assumptions. A timeout can falsely suspect a slow worker; drain and fencing reduce conflicting admissions but do not erase already-issued external side effects. The system does not prove arbitrary model-output correctness or protect against a malicious buyer refusing acceptance.

10.3 Production hardening priorities

Independent security review of escrow V2 and typed-data domains; contract deployment verification; tested key rotation and recovery; encrypted storage retention monitoring; reproducible cross-host recovery; rate limiting and abuse controls; real provider reconciliation; operational backup of buyer-controlled state; and evidence-bound incident procedures.

11. Reference implementation

Layer	Implementation
Buyer console	React, TypeScript and Vite.
API and coordinator	Node.js and TypeScript, job lifecycle, signed action routes, recovery and takeover policy.
Contracts	Solidity 0.8.28 with Foundry; escrow, authority adapter and signing fixtures.
Ledger	SQLite durable payment-operation state and restart reconciliation.
Storage	Arkiv adapter, encrypted manifests and chunked artifacts.
Worker runtime	CLI, heartbeat and acknowledgement clients, local handoff, state adapters and MCP capability probes.
Payment integration	Configurable Hedera x402 provider adapter; local fixture used where external configuration is absent.

11.1 Interfaces and deployment boundaries

The API exposes health, status, protocol, job lifecycle, service-operation and reconciliation routes. A Vite client and Node API may be deployed independently. Secrets for Sepolia, Arkiv, key grants and paid-service endpoints belong in private environment configuration. The worker B environment should not inherit buyer or worker A signing credentials.

12. Verification and evidence ledger

A whitepaper must distinguish source-level implementation, local simulated verification, historical live transactions and current end-to-end demonstration. The repository README reviewed for this document was generated 6 November 2026 and identifies candidate commit a155b5e on the integrated remediation line. Its reported verification included 117 automated tests and 17/17 Foundry tests, with fake-backed or local fixtures for external providers, chain writes and Arkiv. These counts are reported project evidence, not independently reproduced measurements in this document.

Claim	Evidence status as reported
Historical Sepolia lifecycle	17 successful transactions on earlier escrow: A withdrew 3 test tokens, B withdrew 7, takeover reached epoch 2 and stale A was rejected in simulation. Historical V1 baseline only.
ENS scoped authorization	Historical live scoped grant, wrong-job denial and revoke checks reported passed.

Claim	Evidence status as reported
V2 contract behavior	Local tests and compiled source; revised V2 escrow not deployed in the cited baseline.
Encrypted A-to-B handoff	Local synthetic fixture and key-service checks passed; real accepted Arkiv package and separate-host recovery pending.
Arkiv write/read/renewal	Buyer-owned RPC and signer absent in baseline; live proof pending.
Hedera x402 payment	Adapter and policy tests exist; real paid service not configured; live settlement pending.
Automatic takeover	Buyer policy runner implemented and locally tested; live G5 takeover transaction not demonstrated.

The current complete integration should therefore be described as partially verified, not production-proven. A newer repository revision may supersede these results; claims should be refreshed against a pinned commit, deployment addresses and transaction receipts before publication.

13. Demonstration and reproducibility protocol

A persuasive end-to-end test should: (1) pin source commit and V2 escrow address; (2) fund a fresh job; (3) authorize A; (4) execute a real paid operation with durable logical ID; (5) store and read back an encrypted checkpoint and purchased output from Arkiv; (6) accept a milestone; (7) interrupt A; (8) drain and reconcile all in-flight spending; (9) grant B exact-head access on an independent host; (10) validate recovered state and output without another debit; (11) confirm epoch-advancing takeover; (12) reject stale A; and (13) withdraw earned credits and reconcile refunds.

Retain transaction hashes, block numbers, manifest identifiers and digests, payment operation IDs, redacted logs, measured recovery time, spend delta and failure-injection results. Do not publish private keys, decrypted checkpoint secrets or worker credentials. Latency and savings should be reported as measured distributions over repeated runs, not invented point estimates.

14. Comparison and design trade-offs

Alternative	Difference from BATON
Conventional workflow engine	Strong orchestration/retries; BATON adds signed portable authority, escrow commitments and cross-worker recovery boundary, at greater operational cost.
Shared database checkpointing	Simpler for one operator; BATON binds accepted state to independently checkable commitments and scoped successor access.
Fully on-chain state	Transparent but expensive and inappropriate for large or private runtime artifacts; BATON keeps payloads encrypted off-chain.
Unmediated agent handoff	Low friction but weak canonical acceptance, payment reconciliation and stale-worker fencing.

15. Roadmap

Phase I - integrated proof: deploy and verify V2, complete Arkiv read/write and renewal, configure real paid-service status/output lookup, run independent-host B recovery and archive transaction evidence. Phase II - operational resilience: redundant buyer infrastructure, backup/restore drills, stronger key lifecycle, monitoring, formalized incident runbooks and audit. Phase III - protocol generalization: richer capability matching, multiple tasks and independent epochs, configurable verifier attestations, alternative execution adapters and carefully specified decentralization of takeover policy.

16. Conclusion

BATON’s central proposition is that agent work should outlive the process performing it. Its design combines an accepted cryptographic checkpoint, scoped worker authority, epoch fencing, durable payment reconciliation and escrowed incentives into a coherent handoff boundary. The architecture is intentionally hybrid: chain commitments anchor authority and accounting, encrypted off-chain storage carries recoverable state, and buyer-controlled services coordinate safety-critical transitions. The remaining proof obligation is a reproducible, real-provider, real-storage, cross-host V2 lifecycle demonstrating these components together.

References and provenance

[1] BATON source repository and README: https://github.com/noMathematician1729/baton (reviewed 8 November 2026; README status generated 6 November 2026).

[2] BATON audit remediation plan: https://github.com/noMathematician1729/baton/blob/main/plan.md (scope and gate definitions; some planned items are not evidence of completion).

[3] Ethereum EIP-712, Typed Structured Data Hashing and Signing: https://eips.ethereum.org/EIPS/eip-712

[4] ENS documentation: https://docs.ens.domains/

[5] Foundry documentation: https://book.getfoundry.sh/

Editorial note: references to Arkiv and Hedera describe BATON integration intent and code interfaces; no independent sponsor-service performance claims are made.